

Problem 1: Word Embeddings from IIT Jodhpur Data

P1 ▶ Corpus File Size

0.937 MB (982,830 bytes)

Documents: 50 | Tokens: 153,804 | Vocab: 9,970

P1 ▶ Corpus Curation & Preprocessing Steps

- Source: IIT Jodhpur website (Departments, Academics, Research, Faculty profiles, Announcements) using Requests + BeautifulSoup.
- Step 1: Boilerplate Removal - Removing headers, footers, and non-English lines using ASCII character proportion heuristic (>70% ASCII = English).
- Step 2: Text Cleaning - Removing addresses, LaTeX fragments, excessive punctuation, numbers and non-textual content using regular expressions.
- Step 3: Lowercasing - Converting all text to lowercase for vocabulary normalization.
- Step 4: Tokenization - Tokenizing sentences using NLTK's sent_tokenize, then into words using word_tokenize.
- Step 5: Frequency Filtering - Filtering out tokens with frequency < 2 (used as min_count in Word2Vec training) to reduce noise from rare/misspelled tokens.

P1 ▶ Top-10 Words (Frequency-wise)

1. the	6,154	2. of	4,758	3. and	4,423	4. in	2,181	5. to	2,174
6. for	1,805	7. a	1,677	8. ug	1,014	9. iit	951	10. students	

P1 ▶ Interesting Analogy Found

`semester : examination :: thesis : defense`

Interpretation: Just as a semester culminates in an examination, a PhD thesis culminates in a 'defense' (viva-voce). The model also resolved:

`UG : B.Tech :: PG : M.Tech      and      student : learning :: faculty : teaching`

Trained using Word2Vec Skip-gram (vector_size=300, window=5, negative_sampling=5, min_count=2, epochs=20) on the IITJ corpus.

student —

-0.4817, 0.3603, 0.0493, 0.4250, -0.3770, -0.0673, 0.0594, 0.3550, 0.4441, 0.0108, 0.4734,
0.2026, -0.0270, 0.5809, -0.2244, -0.5872, 0.0789, -0.1344, -0.1896, -0.5505, 0.3547,
0.5694, -0.2025, 0.3512, 0.2020, -0.0790, -0.4439, -0.6036, 0.4092, 0.0152, -0.8031,
-0.1061, 0.2068, -0.0710, 0.1819, -0.1978, 0.1547, 0.2107, -0.0920, -0.2565, 0.3691,
-0.9547, -0.2578, -0.0465, 0.6128, -0.1441, 0.2610, -0.0176, -0.5609, 0.1823, -0.1211,
-0.2968, -0.2824, -0.4080, 0.1438, -0.1003, 0.1738, -0.1842, 0.6324, 0.2406, 0.1792,
0.3348, 0.2112, -0.3231, -0.0153, 0.3718, -0.3385, 0.1989, 0.0410, 1.1182, 0.2469, 0.2355,
-0.7243, -0.1846, -0.7783, -0.2690, -0.2737, 0.1185, -0.1831, 0.0040, -0.3267, 0.3263,
0.1461, -0.3609, -0.2161, -0.4403, 0.5465, -0.3295, -0.0014, -0.1480, -0.3047, -0.0644,
-0.4163, -0.2091, -0.4770, -0.1277, -0.3629, 0.3530, 0.2578, 0.4835, 0.2564, 0.3275,
0.7638, -0.5899, -0.1397, -0.1678, 0.3567, -0.2041, -0.4595, -0.1959, -0.0501, 0.2504,
-0.0106, -0.1677, -0.4346, 0.0239, 0.1260, 0.1062, -0.7511, 0.0171, 0.2365, 0.0898,
-0.6174, -0.2059, -0.2231, 0.0275, -0.5134, 0.5352, 0.0127, 0.0232, -0.1253, -0.1755,
0.2097, -0.4404, -0.4178, -0.0805, -0.3315, -0.0120, -0.2659, -0.5955, 0.0000, 0.1635,
-0.2164, -0.0874, -0.1708, -0.0451, 0.7730, -0.0028, 0.0360, -0.3441, 0.0187, -0.3389,
0.2444, 0.3140, -0.2221, -0.4685, -0.4509, 0.3447, 0.2380, -0.0195, -0.1323, -0.0985,
-0.0388, 0.0161, -0.5808, 0.0477, 0.2680, -0.2245, 0.5469, 0.2117, -0.5573, 0.4815,
0.3396, -0.0683, 0.0381, -0.5191, 0.0503, -0.1327, -0.0704, 0.1641, -0.1939, -0.2861,
-0.6366, 0.1738, -0.6682, -0.1642, -0.0663, -0.2858, -0.1228, -0.1536, -0.0461, -0.6755,
-0.1673, -0.1956, -0.3070, -0.3313, -0.1935, 0.1770, 0.0497, -0.4135, 0.5211, -0.6660,
-0.3345, -0.0580, -0.5002, -0.3094, -0.1044, -0.2882, 0.0873, 0.3455, -0.2908, 0.1523,
0.1844, 0.3205, 0.2177, 0.3667, -0.1245, -0.2883, 0.5046, 0.6984, -0.4036, 0.1387, 0.5568,
0.0524, 0.2946, 0.1458, 0.5299, -0.0657, -0.7453, 0.0522, 0.3259, 0.0074, 0.2894, 0.0353,
-0.5590, 0.0846, 0.2497, -0.2185, 0.2897, -0.1043, -0.5683, 0.3164, -0.0470, 0.0958,
-0.2009, -0.5172, -0.0821, -0.0654, 0.6564, -0.8608, 0.1632, 0.1346, -0.0572, 0.0744,
-0.2215, -0.0736, 0.1789, -0.0172, 0.0034, 0.3277, 0.4270, -0.2283, -0.4266, 0.3323,
0.3555, 0.3139, -0.1971, -0.0703, -0.5721, 0.5277, 0.2169, 0.2295, 0.0806, -0.0821,
-0.0810, -0.0473, 0.0258, 0.2180, -1.0748, -0.3741, -0.4136, -0.2667, 0.0858, -0.1651,
0.4323, -0.6172, 0.3687, 0.5480, 0.1269, 0.4493, -0.1455, -0.6407, 0.5454, -0.4102,
-0.3061, 0.0469, 0.3951, 0.2586, -0.2073, -0.1485

Problem 2: Character-Level Name Generation using RNN Variants6

Dataset (Task-0)

1,000 unique Indian names curated from North Indian, South Indian, Bengali, Gujarati, Marathi, and pan-Indian name pools (stored in TrainingNames.txt).

Model Architectures (Task-1) — All implemented from scratch using PyTorch

Model	Architecture Description	Params	Size
Vanilla RNN	Manual VanillaRNNCell (tanh activation). 2 stacked layers. No gating mechanism. Simple recurrence: $h_t = \tanh(W_{ih} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$	61,393	0.24 MB
Bi-LSTM	Manual LSTMCell with 4 gates (i/f/o/g). Forward + Backward passes independently stacked (2 layers each). Outputs concatenated for projection.	442,193	1.69 MB
Attention +RNN	Manual LSTMCell (2 layers) + Luong general attention. At each step: $score = h_t^T \cdot W \cdot h_j$; $context = \sum \alpha_j \cdot h_j$; $output = \tanh(W_c \cdot [context; h_t])$	271,185	1.04 MB

Hyperparameters (shared across all models)

Embedding dim: 32 | Hidden size: 128 | Layers: 2 | Learning rate: 0.003 | Epochs: 100 (Vanilla) / 60 (Bi-LSTM, Attention) | Dropout: 0.2 (Vanilla), 0.4 (Bi-LSTM), 0.3 (Attention) | Optimizer: Adam | LR scheduler: StepLR ($\gamma=0.5$)

Quantitative Evaluation (Task-2)

Model	Novelty Rate	Diversity	Sample Names Generated
Vanilla RNN	24.6%	80.4%	Chaitanya, Namit, Vasuki, Sahan, Shubh, Devi
Bi-LSTM	100.0%	97.6%	Ojw, Yegi, Tookst, Yotas, Gokgs, Hgoy
Att+RNN	100.0%	100.0%	Diverse but less realistic character strings

P2 ▶ Which model works better, and why?

Vanilla RNN performs best for realistic name generation on this dataset.

- Vanilla RNN excels because Indian names follow short, consistent phonotactic patterns (avg 6.1 chars). Its simple recurrence $h_t = \tanh(W_{ih} \cdot x + W_{hh} \cdot h_{t-1})$ captures these local character transitions effectively without over-parameterization.
- Bi-LSTM (442K params) massively over-fitted the 1000-name training set during teacher forcing (training loss $\rightarrow 0.003$) making its autoregressive generation unreliable, since the bidirectional context used during training is unavailable at inference time.
- Attention+RNN had the highest diversity (100%) but generated less realistic names, as the attention mechanism requires longer sequences to learn meaningful alignments; with short names (avg ~ 6 chars) there is insufficient context to attend over.

Conclusion: For short, domain-specific sequences with a small training set, simpler models generalize better than over-parameterized ones.

P2 ▶ Vanilla RNN — Number of Parameters & Model Size

- Trainable Parameters: 61,393
Breakdown: embedding (1,568) + rnn_cell_0.W_{ih} (4,096) + rnn_cell_0.W_{hh} (16,384) + rnn_cell_0.b (128) + rnn_cell_1.W_{ih} (16,384) + rnn_cell_1.W_{hh} (16,384) + rnn_cell_1.b (128) + fc_out.weight (6,272) + fc_out.bias (49)
- Model Size on Disk: 0.24 MB (checkpoint including optimizer state)